Recap

Markov Property

$$\mathbb{P}\{X_{n+1} = j | X_n = i, X_{n-1} = i_{n-1}, ..., X_0 = i_0\} = \mathbb{P}\{X_{n+1} = j | X_n = i\}$$

for all $i_0, i_1, ..., i_{n-1}, i, j$, and all $n \geq 0$

One-Step-Transition Matrix

$$\begin{array}{c} \\ \mathbf{A} \\ \mathbf{B} \\ \mathbf{C} \end{array} \begin{array}{c} \begin{array}{ccc} \mathbf{A} & \mathbf{B} & \mathbf{C} \end{array} \\ \begin{bmatrix} 1/2 & 1/2 & 0 \\ 1/2 & 0 & 1 \\ 0 & 1/2 & 0 \end{bmatrix} \end{array} = M$$

Limiting Distribution:
The limiting distribution is the probability distribution to which the Markov chain converges as the number of steps goes to infinity, if such a distribution exists.

$$\lim_{t \to \infty} \mathbb{M}^t \mathbf{p} = \boldsymbol{\pi}^*$$

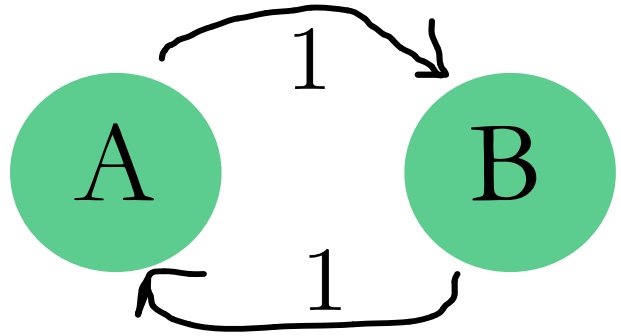
where $\boldsymbol{p}$ is any initial distribution (over all states)

Recap

Stationary Distribution:
A probability distribution $\boldsymbol{\pi}$ is said to be a stationary distribution for a Markov chain if it satisfies the equation:

$$\boldsymbol{\pi} = \mathbb{M} \cdot \boldsymbol{\pi}$$

If you start the process with the stationary distribution $\boldsymbol{\pi}$, then the probabilities of being in each state remain unchanged after any further steps of transitions



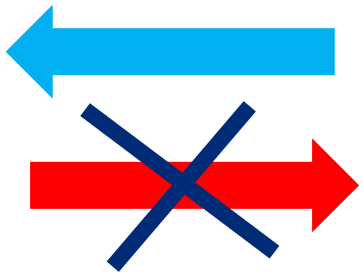
Stationary Distribution

$$\boldsymbol{\pi} = (1/2, 1/2)^T$$

No Limiting Distribution

Stationary Distribution

$$\boldsymbol{\pi} = \mathbb{M} \cdot \boldsymbol{\pi}$$



Limiting Distribution

$$\lim_{t \to \infty} \mathbb{M}^t \mathbf{p} = \boldsymbol{\pi}^*$$

For a Finite-State, aperiodic Markov Chain with all states being accessible from all states, then stationary distribution is equivalent to limiting distribution

Recap

Long-Run Average Distribution (Long-Run Proportion of Time)

This refers to the fraction of time, as the number of steps goes to infinity, that the Markov chain spends in each state *i*.

$$\lim_{t\to\infty} \frac{1}{t} \sum_{k=0}^{t-1} \mathbb{P}\{X_k = i\}$$

Where X_k is the state of the Markov chain at step *k*.

For a Finite-State Markov Chain with all states being accessible from all states

Stationary Distribution ⟺ **Long-Run Average Distribution**

Ways to derive the stationary distribution

1. Directly Solving Equations

$$\boldsymbol{\pi} = \mathbb{M} \cdot \boldsymbol{\pi}$$

2. Computing the Limits

$$\lim_{t\to\infty} \mathbb{M}^t \mathbf{p} = \boldsymbol{\pi}^*$$

3. Simulate the Process long enough and compute

$$\lim_{t\to\infty} \frac{1}{t} \sum_{k=0}^{t-1} \mathbb{P}\{X_k = i\}$$

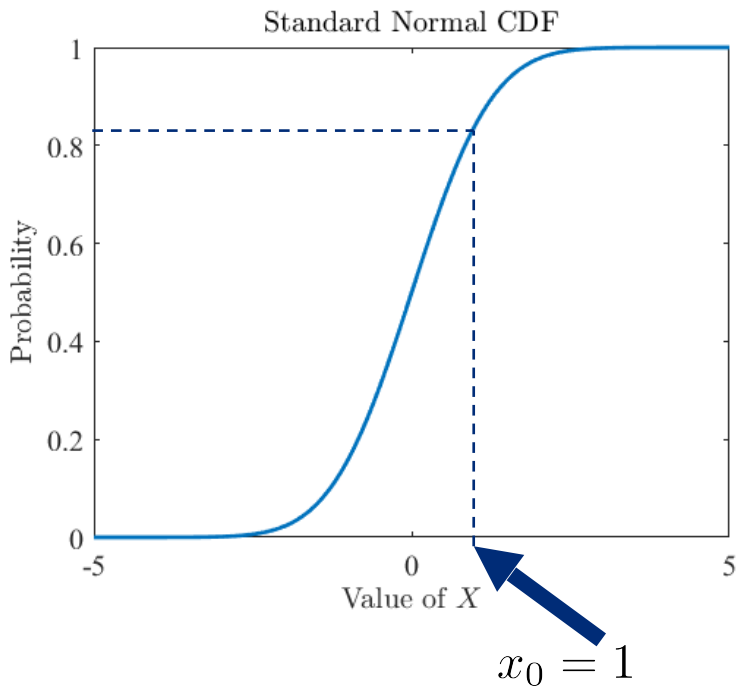
Generate Random Variables: Review of Probability Theory

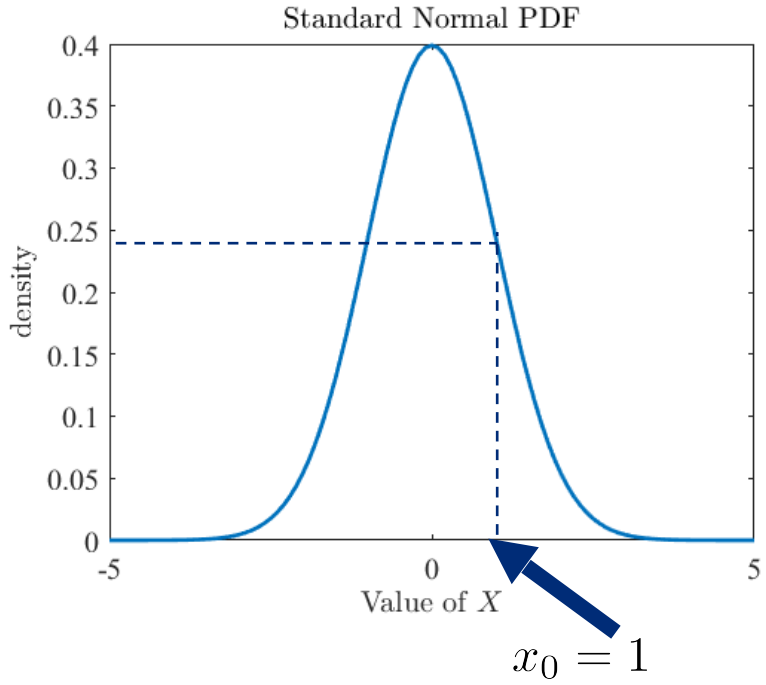
Random Variable: the value is uncertain ex ante

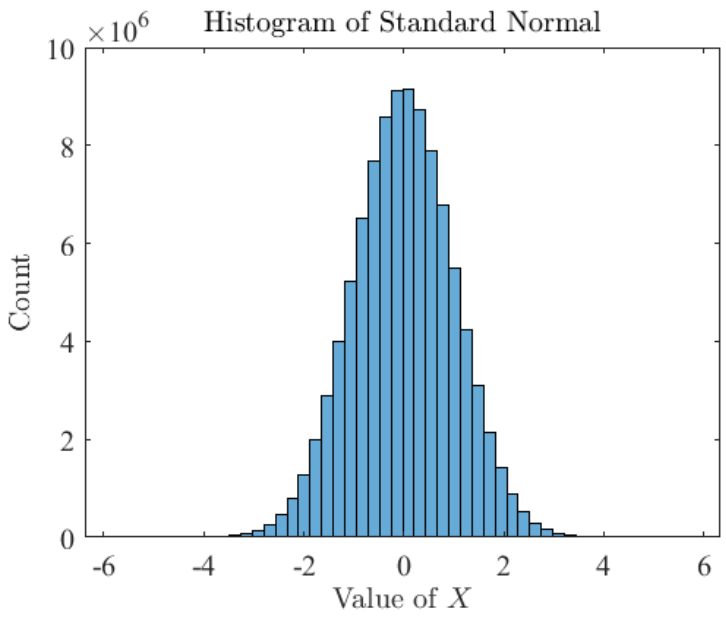
Every random variable has a unique cumulative distribution function (cdf)

$$F_X(x) \triangleq \mathbb{P}\{X \leq x\}$$

The CDF completely determines the distribution of a random variable. The PDF, mean and variance can be derived from it when they exist.

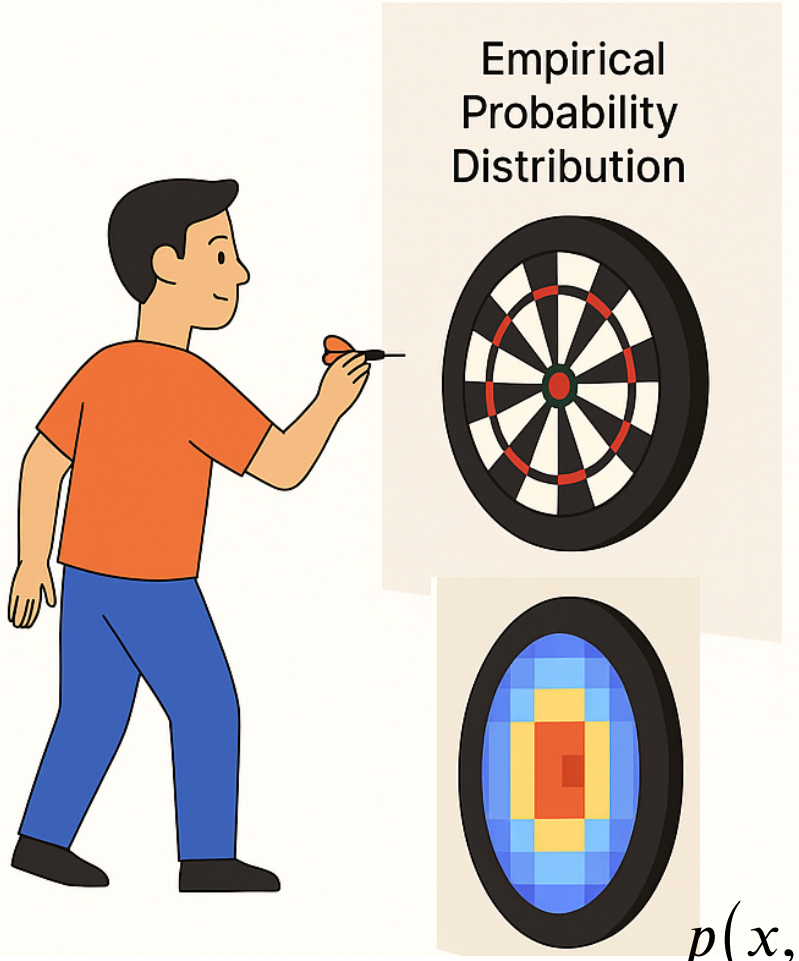




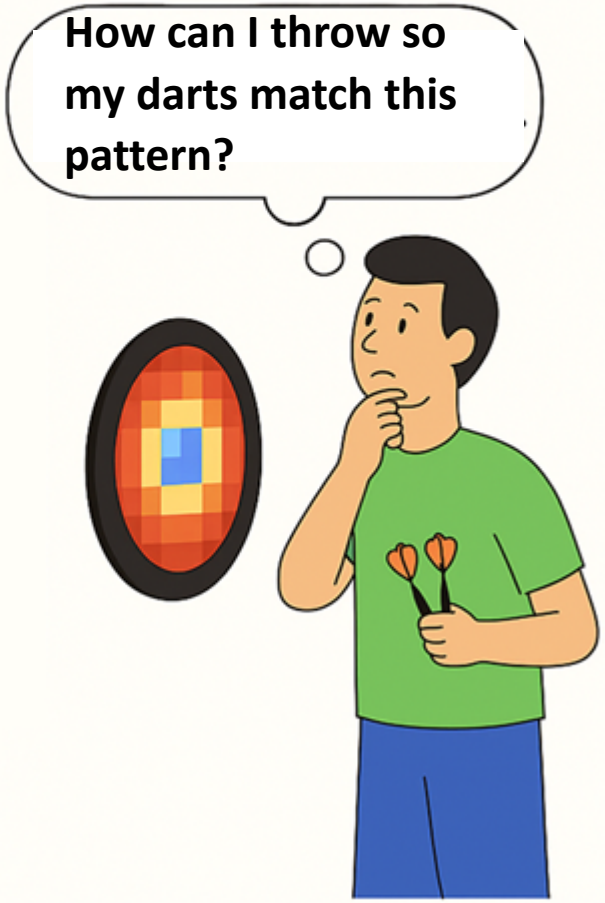


Generate Random Variables: A Reverse Engineering Problem

From Random Throws to Probability Distribution



From Distribution to Random Throws



$$p(x, y) = prob.\ density\ of\ dart\ landing\ positions\ at\ (x, y)$$

Generate Random Variables: Inverse CDF

Suppose we have an algorithm, which takes a single numerical input u and produces a single numerical output x

$$x = algorithm(u)$$

The *algorithm()* is a deterministic input–output transformation,

e.g., *algorithm(0.3) = 2, algorithm(0.9) = 103.45*

Suppose that the input u is not specified, but instead is a random variable U, then the *algorithm(U)* defines a random variable X = *algorithm(U)*

If you know the distribution of U, i.e., $F_U(u)$, then you can compute the distribution of X based on $F_U(u)$ and the deterministic *algorithm()*

Summary: with a known random variable and a deterministic *algorithm()*, you can produce a new random variable

Generate Random Variables: Inverse CDF

Summary: with a known random variable and a deterministic *algorithm()*, you can produce a new random variable

E.g., U is a dice, i.e., $\mathbb{P}\{U = i\} = 1/6, \ \forall\, i \in \{1, 2, 3, 4, 5, 6\}$

$$x = algorithm(z) = \begin{cases} 1, & \text{if } z \geq 4, \\ 0, & \text{if } z < 4. \end{cases}$$

$$\Rightarrow \ \mathbb{P}\{algorithm(U) = 1\} = 1/2 = \mathbb{P}\{algorithm(U) = 0\}$$

So far it's all about forward thinking: you have a random variable U whose distribution is known

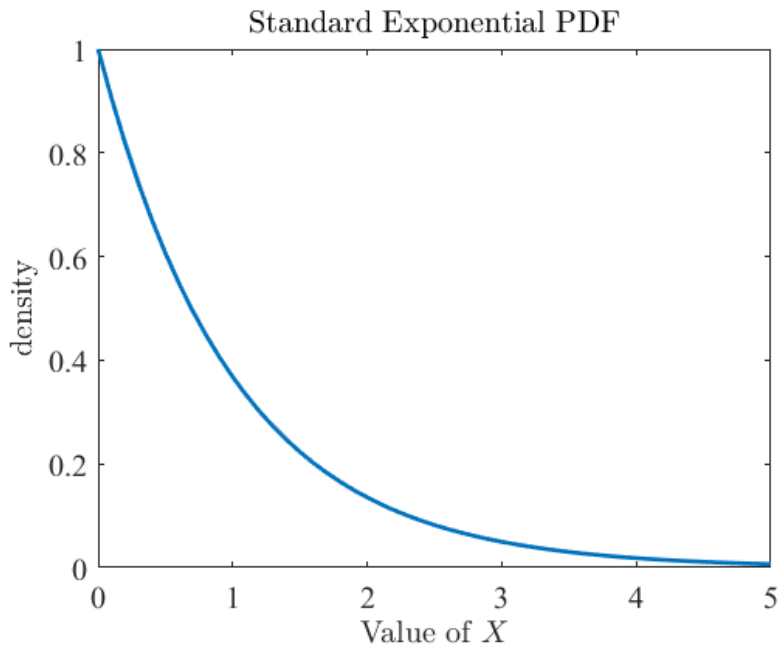
$$X = algorithm(\ \ )$$

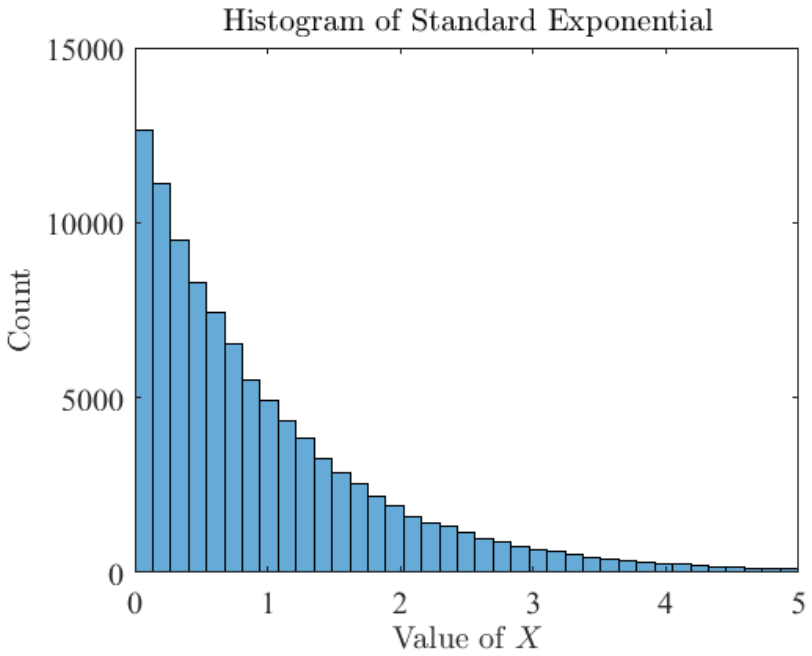
Now let's think the problem backwardly: if I know what kind of random variable I want, how should I generate realizations of this random variable?

Generate Random Variables: Inverse CDF

Let's look at the problem from another angle: I want to generate a standard exponential random variable, what should I do?

Exponential random variable also has built-in function. In matlab, you call *exprnd(1)*

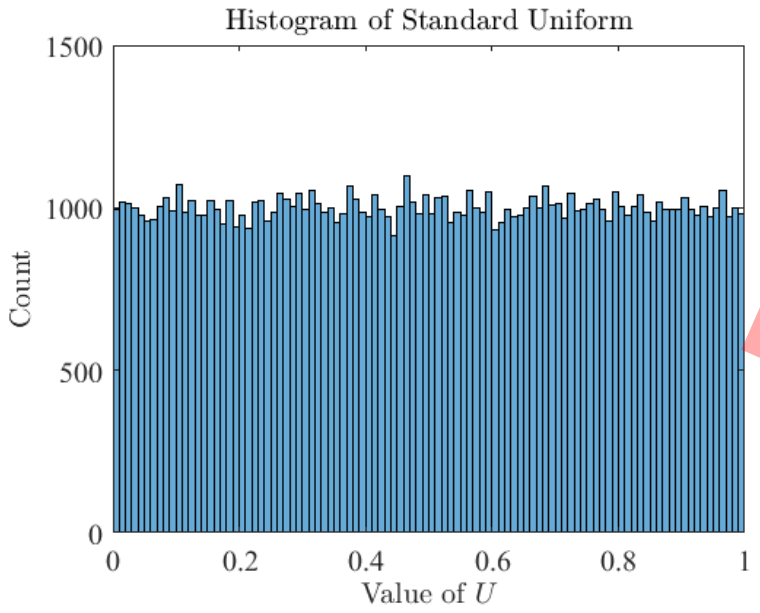




Let's pretend such built-in function does not exists, what should we do?

Generate Random Variables: Inverse CDF---Program exprnd() from Scratch

All you have is a built-in function that generates a standard uniform random variable U

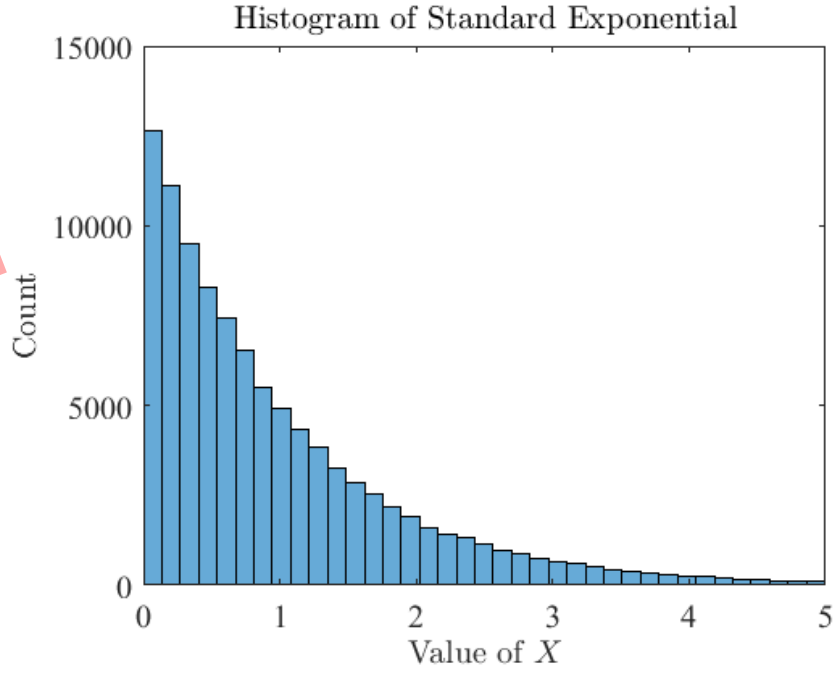


We hope to find a magic algorithm *A()*, such that $X = A(U)$ is an standard exponential random variable

Our goal is to find that magic algorithm *A()* such that:

1. You generate N (large, e.g., 1000000) realizations of U
2. Plug them into $A(U)$ and hence get N values of $A(U)$
3. You plot the histogram of these N values of $A(U)$, it looks like

$A(U) = X$



$$F_U(u) = \mathbb{P}\{U \le u\} = \begin{cases} 0, & u < 0, \\ u, & 0 \le u < 1, \\ 1, & u \ge 1. \end{cases}$$

Generate Random Variables: Inverse CDF---Program exprnd() from Scratch

Forward thinking so far: $U[0,1] + A()$ deterministic $\Rightarrow$ $X \triangleq A(U)$

$$F_U(z) = z, \text{ if } z \in [0,1]$$

$$\Rightarrow F_X(x) = \mathbb{P}\{X \le x\} = \mathbb{P}\{A(U) \le x\} = \mathbb{P}\{U \le A^{-1}(x)\} = F_U(A^{-1}(x)) = A^{-1}(x)$$

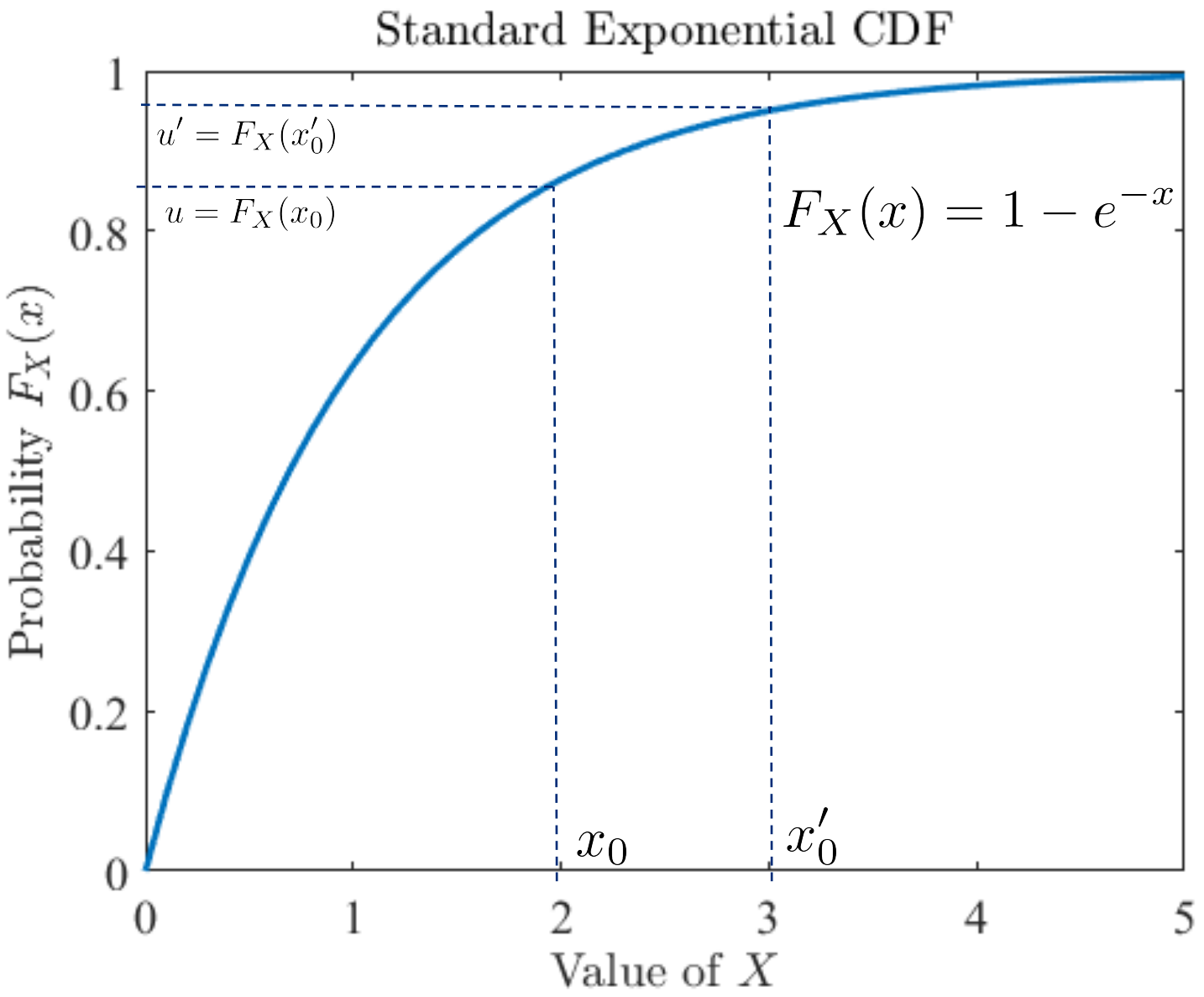
Backward thinking now: I know the functional form of $F_X(x) = 1 - e^{-x}$, and I have a built-in function *rand()* to generate $U[0, 1]$. I just don't know $A()$

Let's try plugging $A(z) = -\ln(1-z)$ to see if the resulting random variable $A(U)$ follows a standard exponential distribution.

$$F_X(x) = \mathbb{P}\{X \le x\} = \mathbb{P}\{A(U) \le x\} = \mathbb{P}\{-\ln(1-U) \le x\}$$
$$= \mathbb{P}\{1 - U \ge e^{-x}\} = \mathbb{P}\{U \le 1 - e^{-x}\} = 1 - e^{-x}, \ x \ge 0.$$

Where does this $A(U) = -\ln(1-U)$ come from?

Generate Random Variables: Inverse CDF---Program exprnd() from Scratch



- Range of ANY cdf is [0,1]
- cdf is always invertible for continuous random variable

$$F_X(x_0) = u \;\Leftrightarrow\; x_0 = F_X^{-1}(u)$$

- There is one-to-one correspondence between the domain and range of $F_X(x)$

$$X = F_X^{-1}(U)$$

- Magic algorithm found $A() = F_X^{-1}()$
- You can use standard uninform random number generator to generate ANY random variables (with cdf)

Generate Random Variables: Inverse CDF---General Case

What if the random variable I want to generate is discrete? E.g., Poisson

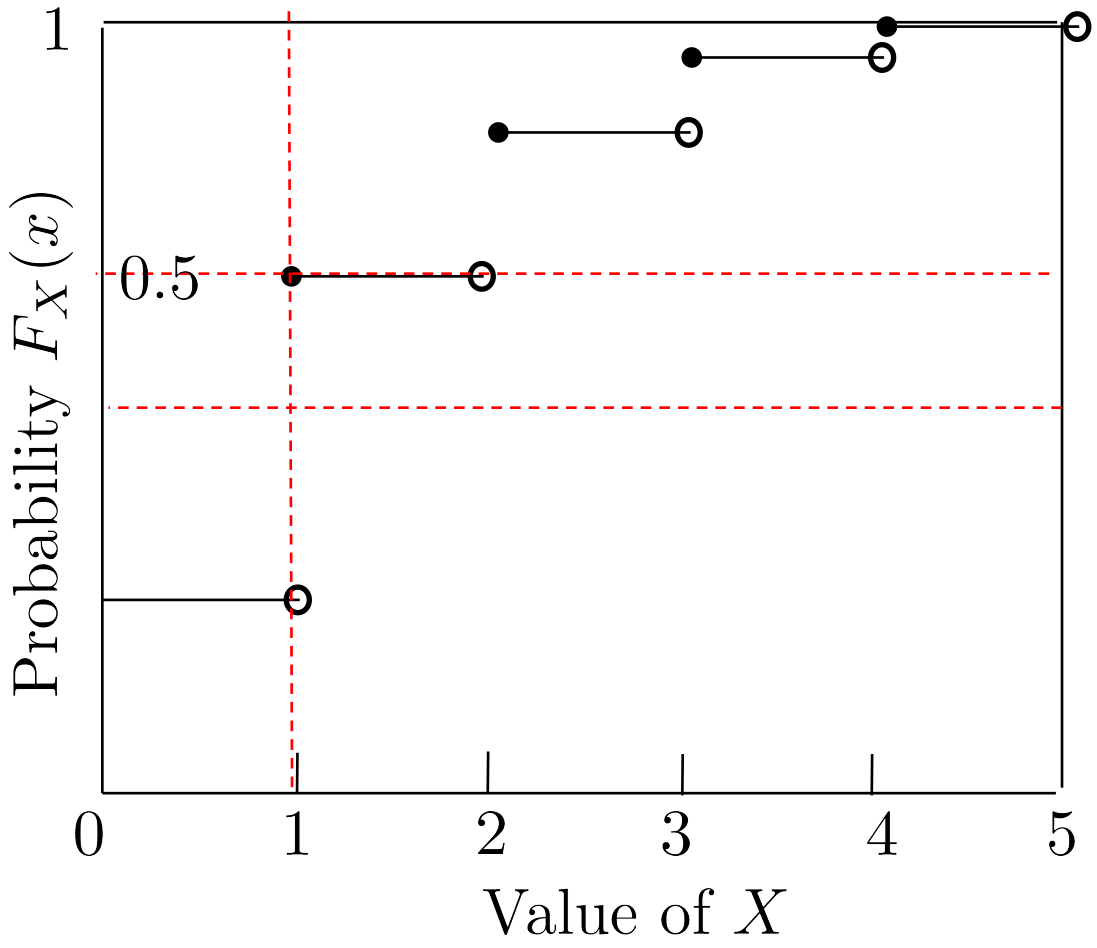
$$\mathbb{P}\{X = k\} = \frac{\lambda^k e^{-\lambda}}{k!}, \ k = 0, 1, 2, 3, \ldots$$

Sometimes I found 0 corresponding value

Sometimes I found many corresponding values

$$x = F_X^{-1}(u) \triangleq \min\{x : F_X(x) \geq u\}$$

We can use standard uninform random number generator to generate any random variables ***theoretically speaking***



Generate Random Variables: Inverse CDF---General Case

Let's try to generate standard normal random variable by inverting its cdf

$$F_X(x) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{x} e^{-t^2/2} dt$$



Recap: Inverting Standard Normal CDF

Inverting a cdf to generate ***ANY*** distribution $x = F_X^{-1}(u) \triangleq \min\{x : F_X(x) \geq u\}$

Let's try to generate a standard normal random variable by inverting its cdf

$$F(x) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{x} e^{-t^2/2} dt$$ Does not have an explicit form

$$u = F(x) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{x} e^{-t^2/2} dt \Leftrightarrow \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{x} e^{-t^2/2} dt - u = 0$$

$\Updownarrow$ We want to write x as a function of u $\Updownarrow$ The solution is unique

$$x = F^{-1}(u) \Leftrightarrow G(x;u) \triangleq F(x) - u = 0$$

$$X = F^{-1}(U) \Leftrightarrow G(X;U) \triangleq F(X) - U = 0$$

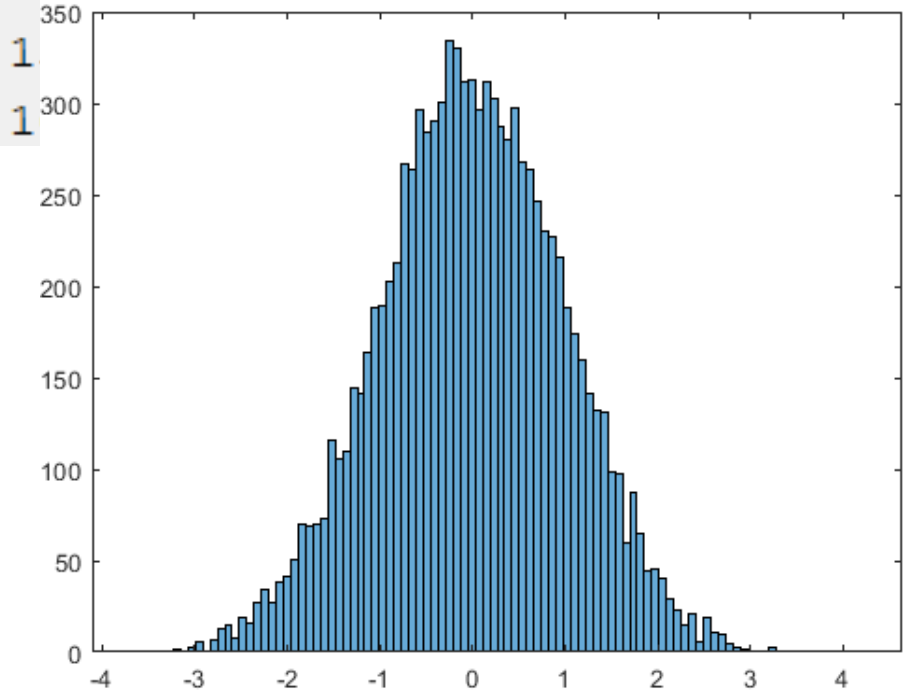
We have transformed the inversion problem into a root-finding problem

Numerically Inverting Standard Normal CDF: Root-Finding Problem

```
6 -  U = rand(10000,1);
7 -  Normal = zeros(10000,1);
8 -  tic
9 -  for i = 1:1:length(U)
10 -     [x,fval]=fzero(@(x) integral(@(t) exp(-t.^2/2),-Inf,x)/sqrt(2*pi)-U(i),0.5);
11 -     if abs(fval)>0.0001
12 -        error('Not root')
13 -     end
14 -     Normal(i) = x;
```

Find root

Initial starting root



$$F(x) - u = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{x} e^{-t^2/2} dt - u = 0$$

Computational cost in this implementation

- Numerical Integration
- Numerically find root

→ Slow

Another Method of Generating Random Variables: Acceptance-Rejection

Let X be the random variable of interest. The rejection method applies when we can express

$$F(x) \triangleq \mathbb{P}\{X \leq x\} = \mathbb{P}\{V \leq x | \mathcal{A}\}$$

where V is another random variable and A is an "acceptance" event related to V

This suggests that we could use the following algorithm to generate X

```
Step 1. Generate V
Step 2. Check if event A occurs
        Yes,return X = V;
        no, go to Step 1.
```

```
U = rand(10000,1);
X = zeros(10000,1);
for i = 1:1:length(U)
    X(i) = invXCDF(U(i))
end
```

***N* in, *???* out** ***N* in, *N* out**

The acceptance-rejection method works well when there are not too many rejections.

It is better than the inverse cdf when the cdf is hard to invert and there are not too many rejections.

Acceptance-Rejection: A Modification of Inverse CDF Method

In order to use some basic random variable generator to generate new random variables, two things are need to be satisfied:

(1) Output of the transformation must coincide with the support of the target r.v. X

Support of a random variable: the set of possible values of a random variable

Many functions can achieve this goal: map (0,1) to the real line:

$$f(x) = \ln(1/x - 1) \qquad f(x) = \ln(2/x^2 - 2)$$

Which function to choose?

(2) Output of the function must have same statistical properties as the target r.v. X

Statistical properties of a random variable is uniquely determined by its cdf

This why inverse cdf method works

Plugging standard uniform random number generator into the inverse cdf is not the only way to make it work

Carefully design the acceptance condition can also make it work

Cdf of a random variable tells you which part of the support is dense or sparse

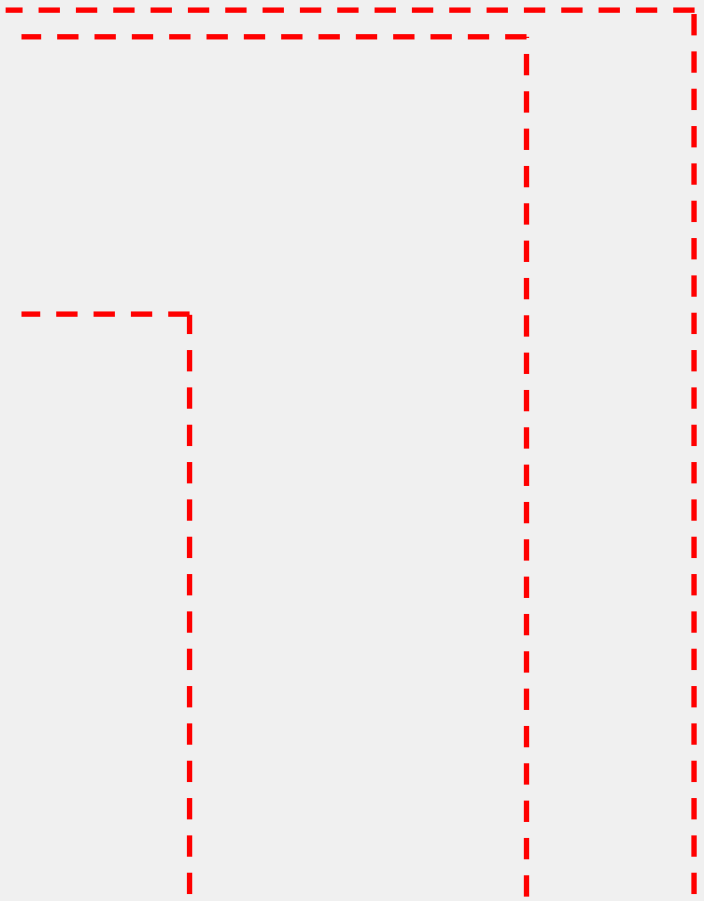
Cdf is a way to assign pre-specified weights on different region of the support of a r.v.

Examples (Baseline)

Using a standard uniform r.v. to generate another standard uniform r.v. $U[0,1] \mapsto U[0,1]$

Example

Using standard uniform to generate standard exponential: $U[0,1] \mapsto \exp(\lambda = 1)$

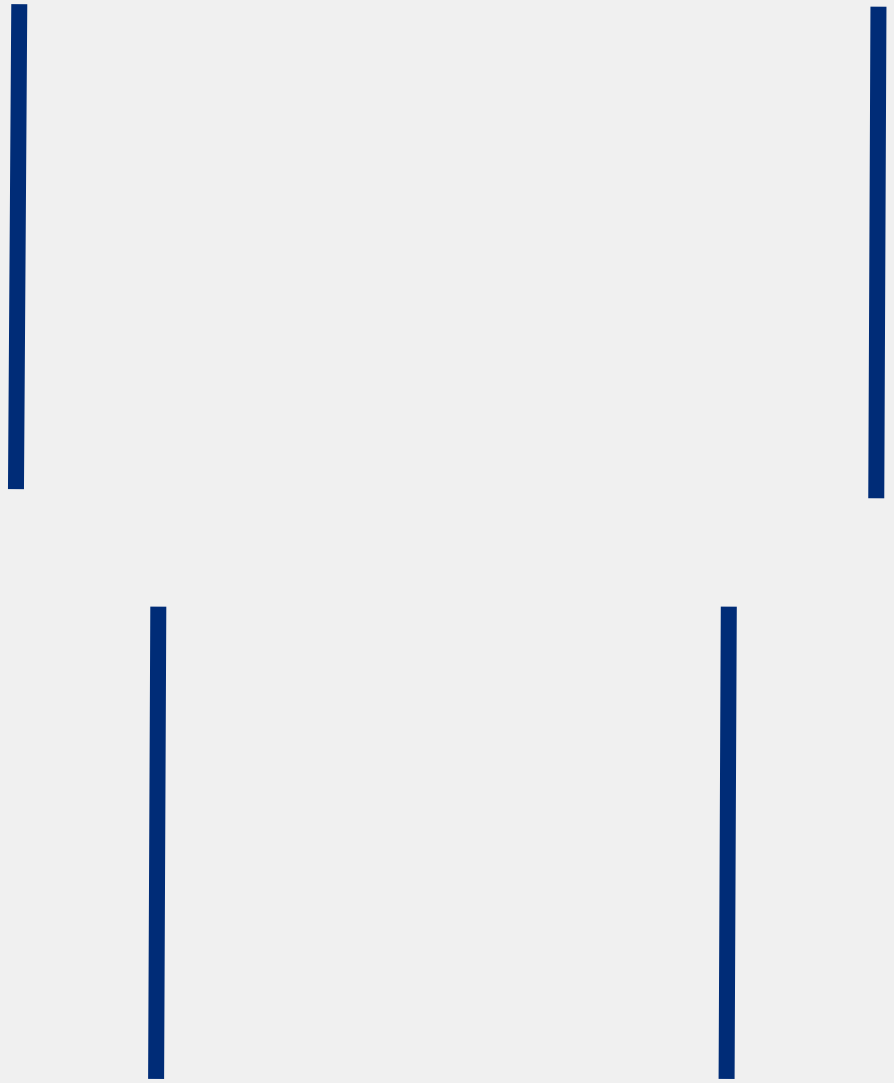


The only difference between this gif and the previous one is the mapping function

[0, 2] is dense and [5, 10] is sparse

Example

Using standard uniform to generate standard logistic and standard normal



$$F(x) = \frac{1}{1+e^{-x}}$$

$$f(x) = \frac{e^{-x}}{(1+e^{-x})^2}$$

Stretched version of "S"

$$F(x) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{x} e^{-t^2/2} dt$$

$$f(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}$$

Summary

By far, you should know:

How the random variables are generated by inverse cdf method?

Why we say cdf uniquely characterizes a random variable?

- It tells you about which part in the support will be dense (or sparse) if you keep on generating many realizations of this random variable (It specifies the probability assigned to every interval)

What is the role of the standard uniform random variable?

- Regard it as the engine of all the randomness

Now, one question remains: what is the connection between inverse cdf and rejection method?

Connection between Inverse CDF and Acceptance-Rejection Method

If you have two random variables and want to make them "look similar"

you can squeeze and/or stretch the cdf of one random variable, which changes the density on the support

e.g., from standard logistic random variable to standard normal random variable

Standard normal is more concentrated than standard logistic; standard normal has less extreme value than standard logistic

- If you see large abs. value of standard logistic random variable (e.g., >4 or <-4), reject it with high probability
- If you see small abs. value of standard logistic random variable, then accept it with high probability

Because we may reject some realizations of the original random variable, we need to generate more realizations of the original random variable

This changes the density on the original support. By carefully designing the rejection condition, we can modify one cdf to make it look like another random variable's cdf

Implementation of Acceptance-Rejection Method

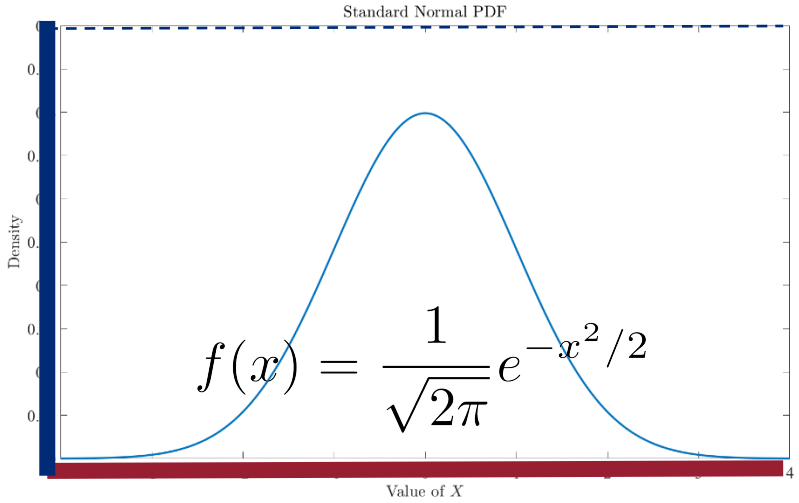
When we implement acceptance-rejection method, we work with density function (pdf)

- cdf: inversing method
- pdf: acceptance-rejection method

Target: generate standard normal random variables

All we need is two independent non-standard uniform random variable generators

$U_2[0, 0.5]$

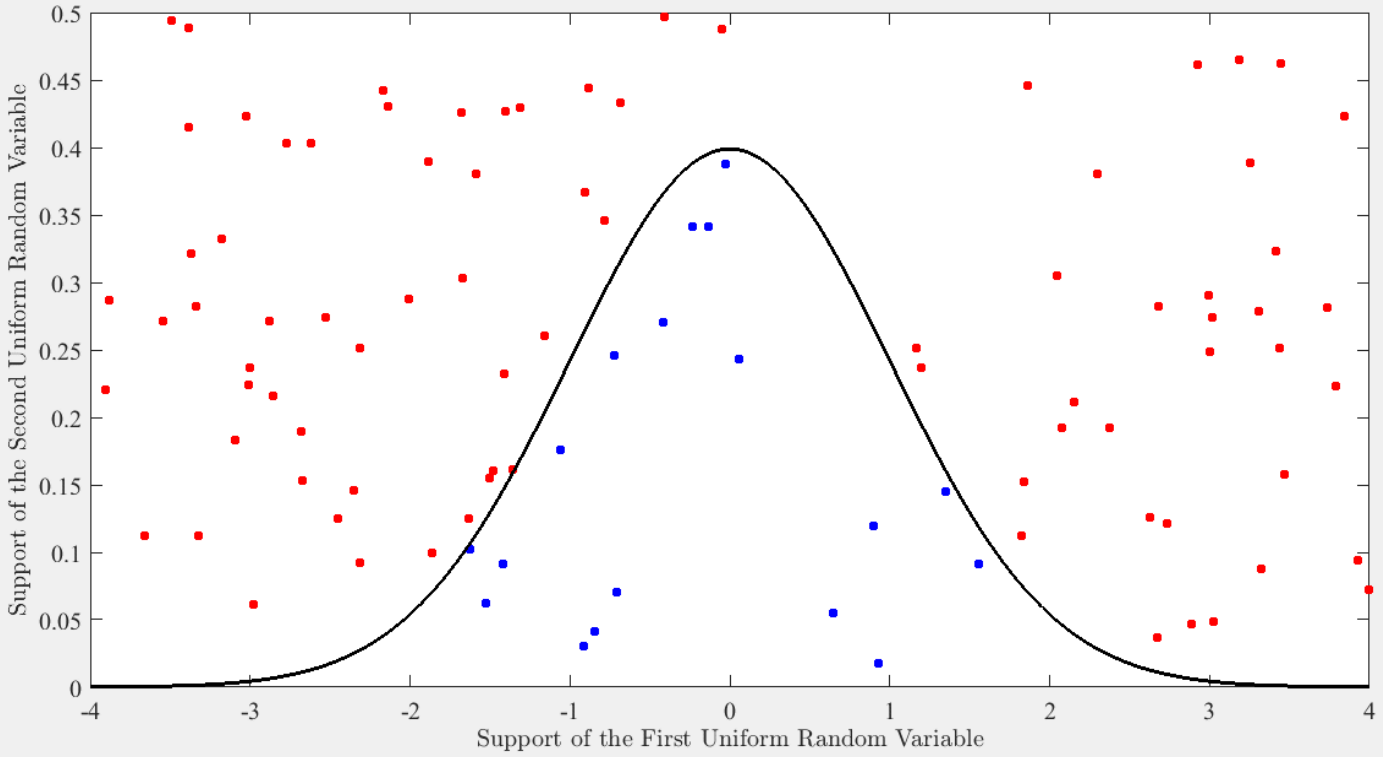


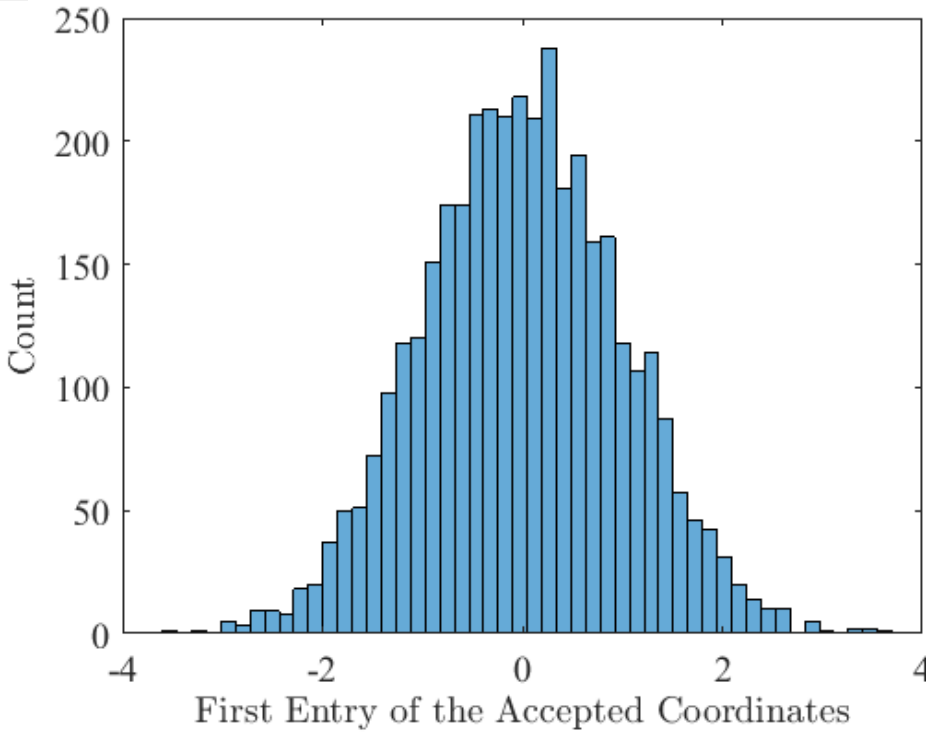
$U_1[-4, 4]$

$$U[a, b] = a + (b - a)U[0, 1]$$

- Continuously generate coordinates (U_1, U_2)
- Accept the points that below the pdf ($U_2 \leq f(U_1)$)
- Collect U_1 in the accepted coordinates

Implementation of Acceptance-Rejection Method

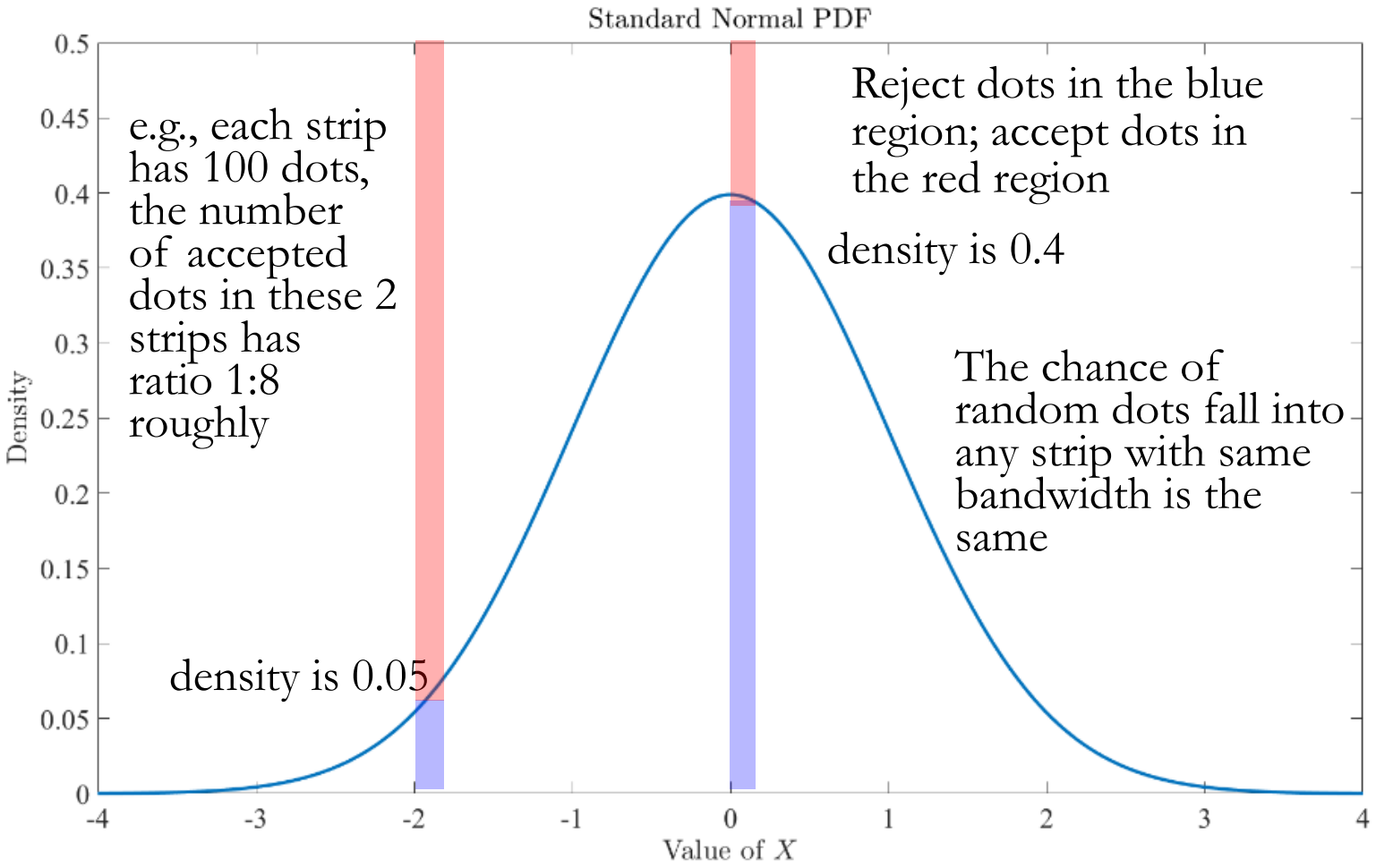




PDF is a "Filter"

Why the 1st entry of those accepted points come from standard normal distribution?

In essence, distribution function (cdf or pdf) tells you which part of the support is dense and which part of the support is sparse





Michelangelo: ***"It's simple. I just removed everything that is not David."***



Advantages of Acceptance-Rejection Method

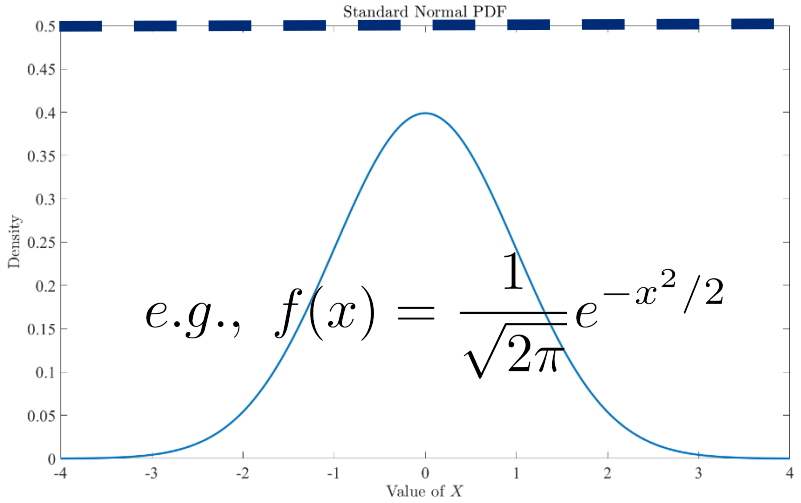
The implementation does not need much math derivation

To generate realizations of a random variable, only need

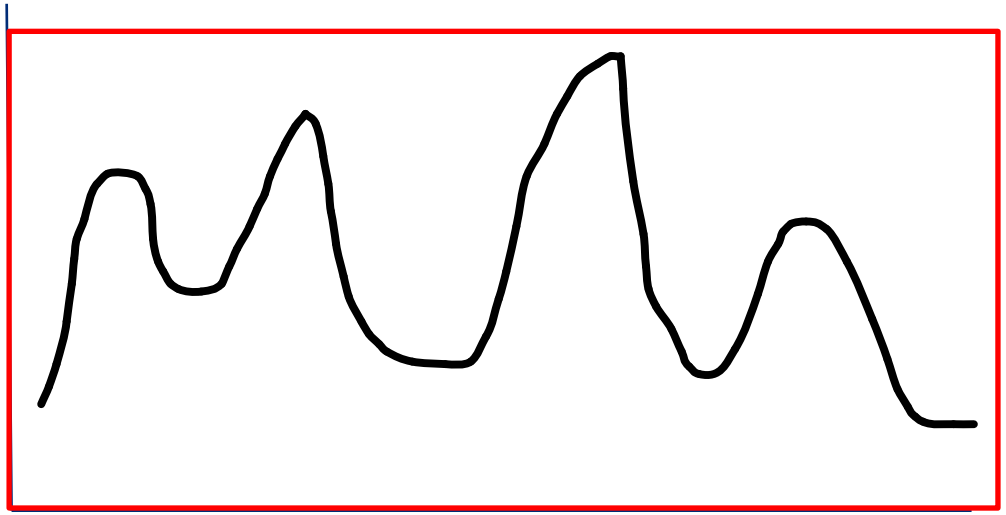
- Two independent uniform (non-standard) random number generators
- Pdf of the target random variable

Need to make sure that the pdf is easy to evaluate at each point

Try to reject less original points



Can handle "irregular" distribution



Potential issues: inefficiency if the target distribution has multiple peaks, and in high dimensions

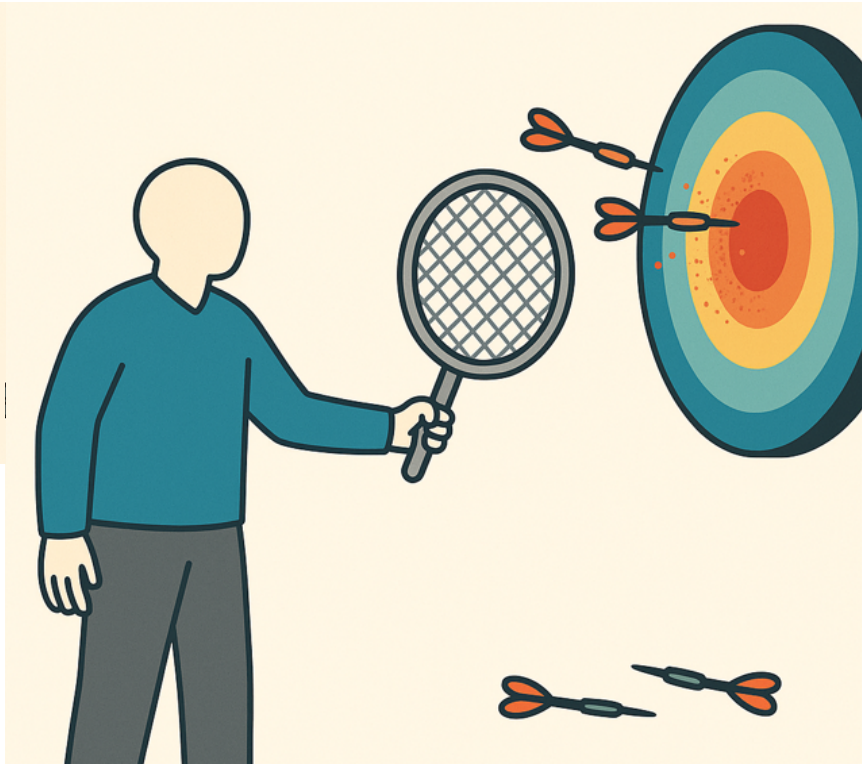
Dart Throwing Analogy: “Before vs. After the Throw”

Inverse CDF — “Transforming randomness before the dart leaves your hand”



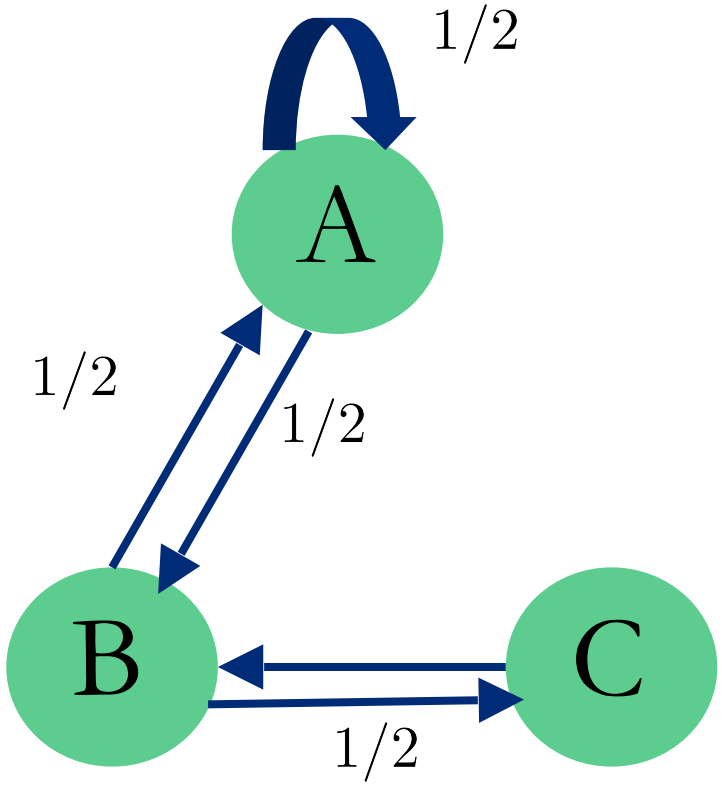
Acceptance–Rejection — “Filtering randomness after the dart leaves your hand”



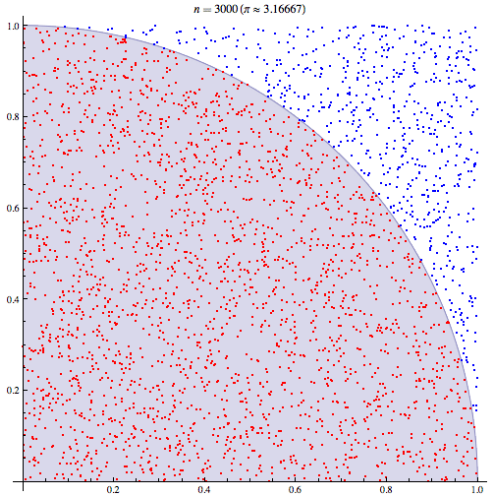


Markov Chain Monte Carlo (MCMC)

Markov Chain:



$$\begin{array}{c} \\ \mathbf{A} \\ \mathbf{B} \\ \mathbf{C} \end{array} \begin{array}{c} \begin{array}{ccc} \mathbf{A} & \mathbf{B} & \mathbf{C} \end{array} \\ \begin{bmatrix} 1/2 & 1/2 & 0 \\ 1/2 & 0 & 1 \\ 0 & 1/2 & 0 \end{bmatrix} \end{array} = M$$



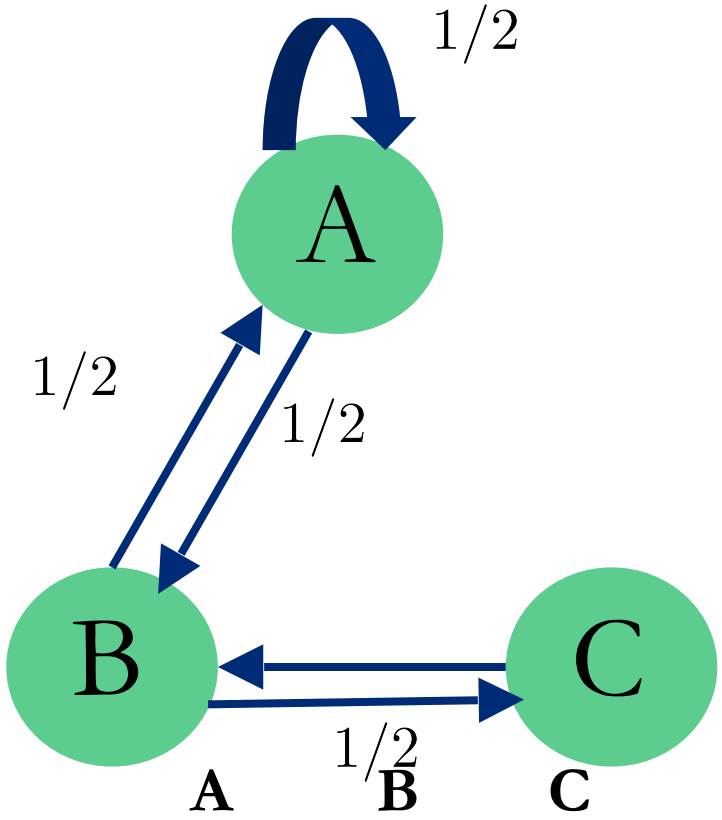
Monte Carlo Method:

Generating realizations of random variables to estimate quantities

I have a Markov Chain, I have a Monte Carlo — ah! Markov Chain Monte Carlo!

Markov Chain Monte Carlo (MCMC)

Markov Chain:



$$\begin{array}{c} \\ \mathbf{A} \\ \mathbf{B} \\ \mathbf{C} \end{array} \begin{bmatrix} 1/2 & 1/2 & 0 \\ 1/2 & 0 & 1 \\ 0 & 1/2 & 0 \end{bmatrix} = M$$

Stationary Distribution ⟵ **Limiting Distribution**

$$\boldsymbol{\pi} = \mathbb{M} \cdot \boldsymbol{\pi}$$

$$\lim_{t \to \infty} \mathbb{M}^t \mathbf{p} = \boldsymbol{\pi}$$

$$X_1, X_2, \cdots, X_{1000}, X_{1001}, X_{1002}, \cdots$$

Approximately Stationary

Can we construct a Markov Chain, such that its Stationary Distribution equal to the target distribution that we want to sample from?

Markov Chain Monte Carlo (MCMC)

$$x_1 = x_0 + \mathcal{N}(0, \sigma^2)$$

$$\Updownarrow$$

$$x_1 = \mathcal{N}(x_0, \sigma^2)$$

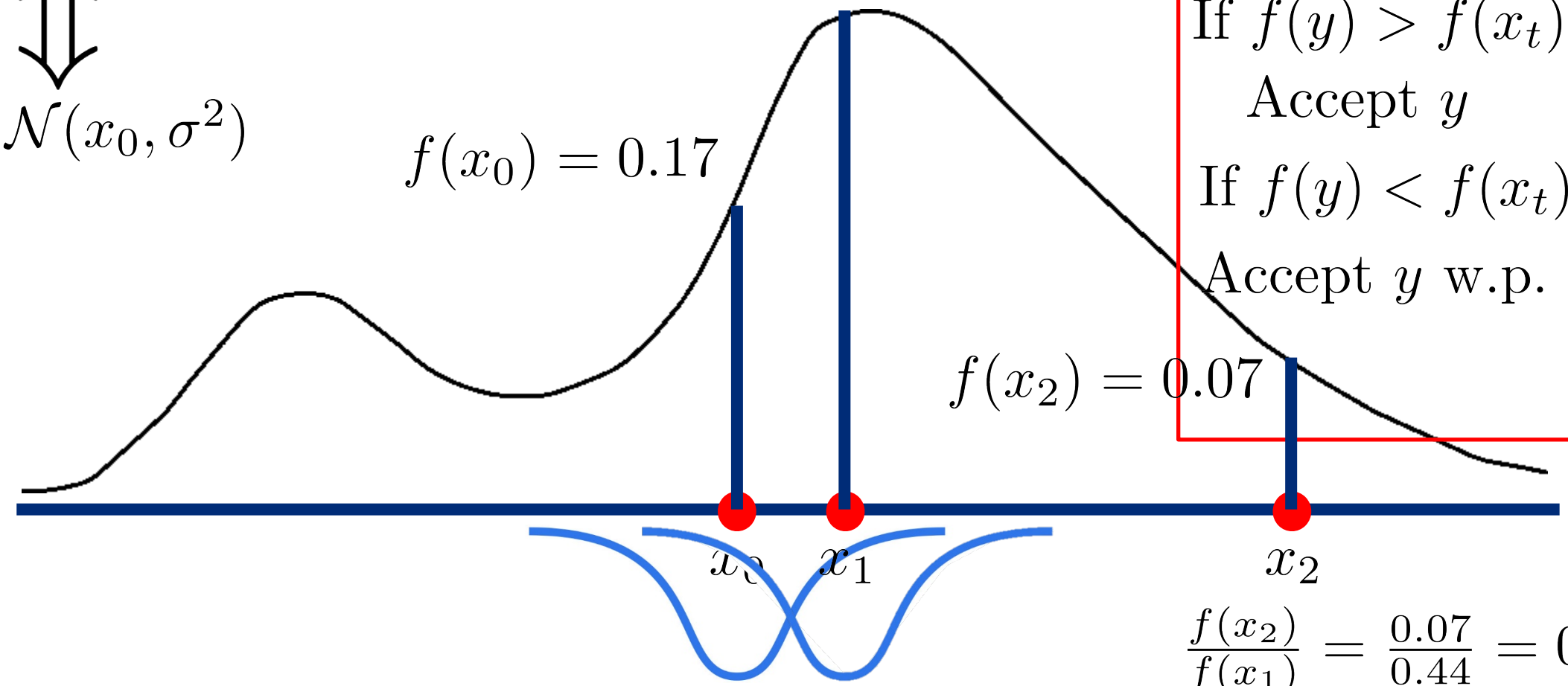
Metropolis Acceptance Rules

If $f(y) > f(x_t)$:

Accept y

If $f(y) < f(x_t)$:

Accept y w.p. $\frac{f(y)}{f(x_t)}$



$$\frac{f(x_2)}{f(x_1)} = \frac{0.07}{0.44} = 0.16$$

$$x_2 = \mathcal{N}(x_1, \sigma^2)$$

See if $U[0, 1] < 0.16$